Heap / Priority Queue Review Sheet

A heap is a machine that can "hand you the smallest (or largest) value at any moment." With Python's heapq, you can efficiently solve: repeated extreme-value queries, top-k, priority scheduling, merging k sorted lists, and shortest-path candidates. This book walks from min-heap basics all the way to Dijkstra — connecting directly to Book 05's unweighted shortest path.

Python 3 · heapq | Tutor style | Includes diagrams | Includes FAQ | Dijkstra intro

00 Review Order + The Three Universal Questions

Heap problems look wildly different on the surface, but every single one can be cracked open with the same three questions. Memorize these three first, then work through the problem types in order.

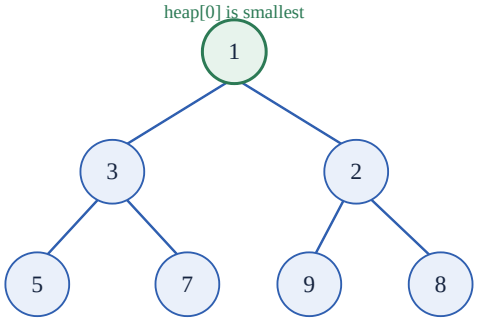
① What's actually	② What does	③ Is the final return
stored in the heap?	**heap[0] represent?**	**what the problem wants?**

01 ← start	02	03	04	05	06
min / max heap	**K / Kth largest**	**Priority Queue**	**Top K / heapify**	**Merge K • K closest**	**Dijkstra**
Extreme values	Maintain size-k	tuple heap	Count + build	Multi-way • distance	Weighted shortest path

Why these three questions actually work: a heap itself is simple (it just "hands you the smallest value") — the difficulty is entirely in what *you* put into it, whether the first element of what you put in is the sort key, and whether you need to process it further after taking it out. 90% of bugs happen at question ③: the heap holds tuples like `(count, number)` or `(distance, point)`, and the problem only wants one part of that — forgetting to extract it is the classic mistake.

01 Heap Concept

A heap is an "almost complete binary tree," but Python stores it as a **list**. It's **not a sorted array** — the only guarantee is that `heap[0]` is always the current minimum (for a min heap).



heap = [1, 3, 2, 5, 7, 9, 8]

Index arithmetic in a list — index

```
        # for a node at index i:
    left  = 2 * i + 1     # left child
   right = 2 * i + 2     # right child
     parent = (i - 1) // 2  # parent
  # property: parent <= both children
# so root (index 0) is the global minimum
```

Compare to the BST you already know: a BST is "left < root < right," which is great for **searching** and gives sorted output via inorder traversal. A heap is "parent <= children," which only keeps the minimum at the top, making it perfect for **repeated extreme-value extraction**. A heap does not sort the elements in between — as long as `heap[0]` is the minimum, it's a valid heap.

02 Min Heap Basics

Three core operations: `heappush` to insert (auto-repositions), `heappop` to pop the minimum (auto-adjusts), and `heap[0]` to peek at the minimum without popping.

The three-piece toolkit — min heap

```python
import heapq

def peek_min(heap):
    if not heap:              # can't read heap[0] on an empty heap!
        return None
    return heap[0]

def pop_all_sorted(heap):     # keep popping the minimum -> naturally ascending
    result = []
    while heap:
        result.append(heapq.heappop(heap))
    return result
```

`Can't index heap[0] on empty heap` `Can't heappop an empty heap` `Check emptiness with if not heap`

Q Is a heap the same thing as a sorted list?

No. It only guarantees that `heap[0]` is the minimum — the order in the middle is unspecified. To get a fully sorted result, you have to "repeatedly pop the minimum" and pull them out one at a time (this is, in fact, exactly what heap sort does).

Q Why is heap[0] guaranteed to be the minimum?

Because heapq maintains the **min-heap property**: every parent node is <= its children. That property holds layer by layer up the tree, which guarantees the root (index 0) is the minimum of the entire tree. Every `heappush` / `heappop` call automatically repairs this property after the operation.

03 K Largest / Kth Largest

Finding the largest k values **doesn't require sorting everything**. The trick: maintain a **min heap of size k**, which always holds "the largest k values seen so far."

Counter-intuitive but key: to find the largest k, you use a **min** heap. Because you only want to keep the largest k in the heap — the moment you exceed k, the one that most deserves eviction is "the smallest among these candidates," and a min heap's `heap[0]` is exactly that — one `heappop` removes it.

kth_largest — maintain a size-k min heap — size-k

```python
def kth_largest(numbers, k):
    if k <= 0 or k > len(numbers):     # the "0th largest" doesn't exist; out-of-range is also invalid
        return None
    heap = []
    for number in numbers:
        heapq.heappush(heap, number)
        if len(heap) > k:              # over k -> evict the smallest candidate
            heapq.heappop(heap)         # ← must be INSIDE the loop!
    return heap[0]   # the heap now holds the largest k; the smallest of those = the kth largest
```

Why does returning `heap[0]` give the kth largest? The heap ends up holding "the largest k numbers," and `heap[0]` is the smallest among those k — which is exactly the overall kth largest. Example: `[5,1,9,3,7,2,8]` with k=3, the largest three are `{7,8,9}`, the smallest of which is 7 → the 3rd largest is 7. To get "the largest k numbers themselves" instead, just swap `return heap[0]` for `return sorted(heap)`.

Q Why does the pop line have to be inside the for loop?

If you write `if len(heap) > k: heappop` **outside** the loop, it only executes once — with 7 numbers and k=3, that's only one pop, leaving 6 in the heap at the end, which is completely wrong. It has to be **checked after every single push** to keep the heap's size pinned at k. Indentation decides correctness here.

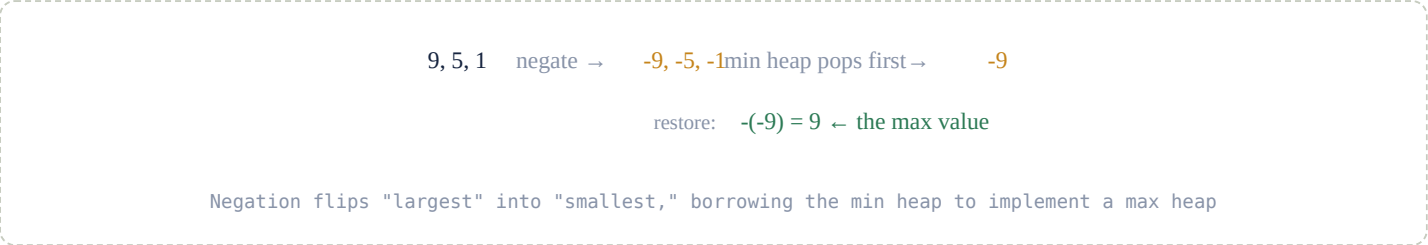
Q Why is the boundary check `k <= 0` instead of `k < 0`?

Because "the 0th largest" doesn't exist either — k=0 is also an invalid input. Writing only `k < 0` would miss the k=0 case, so it has to be `k <= 0`.

Complexity — a direct `sorted` call is O(n log n); maintaining a size-k heap is **`O(n log k)`**. When k is much smaller than n, the heap is noticeably more efficient.

04 Max Heap (Negation Trick)

heapq only supports min heaps. To simulate a max heap, **negate values going in, negate them again coming out**. The largest number, once negated, becomes "the smallest negative number," which a min heap will pop first.



build / pop max heap — max heap

```python
def push_max(heap, number):
    heapq.heappush(heap, -number)     # store negated

def pop_max(heap):
    if not heap:
        return None
    return -heapq.heappop(heap)        # negate again after popping, restoring the positive value
```

The most common mistake: writing `return heapq.heappop(heap)` and returning that directly — what comes out is a negative number like `-9`, and you've forgotten to negate it back. Remember: **add the minus sign going in, add it again coming out** — the two negations always appear as a pair.

05 Priority Queue

A plain queue is first-in-first-out; a priority queue is **highest-priority-out-first**. The technique: store **tuples** `(priority, data)` in the heap — heapq compares tuples by their 0th element first, so it automatically sorts by priority.

PriorityTaskQueue — smaller priority value comes out first — pq

```python
class PriorityTaskQueue:
    def __init__(self):
        self.heap = []

    def add_task(self, task_name, priority):
        heapq.heappush(self.heap, (priority, task_name))  # priority goes first!

    def pop_task(self):
        if not self.heap:                  # check emptiness with not, not is None
            return None
        priority, task_name = heapq.heappop(self.heap)     # unpack to extract the data
        return task_name

    def is_empty(self):
        return not self.heap
```

Q Would storing the tuple as `(task_name, priority)` work?

No. heapq compares tuples by looking at **element 0 first**. Writing `(task_name, priority)` would sort **alphabetically by task name**, not by priority. To sort by priority, priority must be first: `(priority, task_name)`.

Q Why can't you check for an empty queue with `if self.heap is None`?

An empty heap is `[]`, **not** `None`. `[] is None` evaluates to False, so the code would continue on and call `heappop` on an empty heap → crash. Always check for an empty container with `if not self.heap` (an empty list is itself falsy).

Connects back to Book 01 — this is the upgraded version of a plain queue: a queue dequeues by "arrival order," a priority queue dequeues by "priority," with a heap swapped in underneath.

06 Top K Frequent

A two-step combo: **dict counting (Book 03) + heap for top-k (this book)**. The heap stores `(count, number)` — the least-frequent candidates get evicted, leaving the k most frequent at the end.

top_k_frequent — count + heap

```
def top_k_frequent(numbers, k):
    if k <= 0 or not numbers:
        return []
    counts = {}
    for number in numbers:
        counts[number] = counts.get(number, 0) + 1   # counting: must write the result BACK into the dict!
    heap = []
    for number, count in counts.items():
        heapq.heappush(heap, (count, number))         # store (count, number)
        if len(heap) > k:
            heapq.heappop(heap)                        # evict the least-frequent candidate
    return [number for count, number in heap]         # extract just number, don't return raw tuples
```

Q Why doesn't `counts.get(number, 0) + 1` alone work?

That line only **computes** the new value — it never **stores** it back into the dict. It has to be written as `counts[number] = counts.get(number, 0) + 1` — the assignment on the left is what actually updates the count (an old trap from Book 03's counting template).

Back to the three universal questions: ① the heap stores `(count, number)`; ② `heap[0]` is the current candidate with the fewest occurrences; ③ the problem wants number, so the final line `[number for count, number in heap]` extracts it — returning the raw heap directly would give a pile of tuples, which is wrong.

07 Heapify (Build In Place)

`heapq.heapify(lst)` turns a plain list into a valid heap **in place**, in only O(n) — faster than pushing elements one at a time, which is O(n log n).

heapify — note that it mutates the original list — heapify

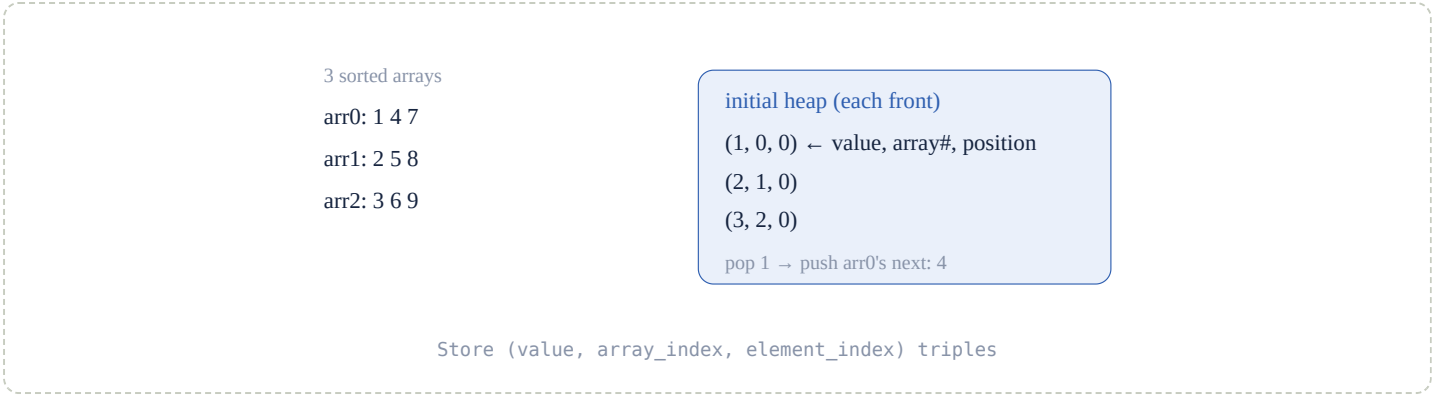
```
def get_sorted_by_heapify(numbers):
    heap = numbers.copy()     # copy first! otherwise you'll mutate the original data
    heapq.heapify(heap)       # O(n) in-place heap construction
    result = []
    while heap:
        result.append(heapq.heappop(heap))
    return result
```

Q Why does `heap = numbers` followed by heapify end up corrupting the original data?

Because `heap = numbers` is just **an alias** — both names point to the **same list**. heapify rearranges in place, so the original `numbers` gets scrambled right along with it. To protect the original data, you must `heap = numbers.copy()` first, then operate on the copy.

08 Merge K Sorted Arrays

Merging k sorted arrays. The heap only ever holds **the current front element of each array** — every time you pop one, you push in **the next element from that same array**.



merge_k_sorted_arrays — k-way

```python
def merge_k_sorted_arrays(arrays):
    if not arrays:
        return []
    heap = []
    for array_index, array in enumerate(arrays):
        if array:                                   # skip empty arrays! otherwise array[0] crashes
            heapq.heappush(heap, (array[0], array_index, 0))
    result = []
    while heap:
        value, array_index, element_index = heapq.heappop(heap)
        result.append(value)
        next_index = element_index + 1
        if next_index < len(arrays[array_index]):  # if this array has more elements, push the next one in
            next_value = arrays[array_index][next_index]
            heapq.heappush(heap, (next_value, array_index, next_index))
    return result
```

Why store three values? After popping a value, you need to know **which array it came from** (array_index) and **which position it was at** (element_index), in order to push the next element from that same array into the heap. Value is stored first so the heap sorts by numeric value. Empty sub-arrays must be skipped — real-world data can look like `[[], [1,3], [], [2,4]]`.

09 K Closest Points

Find the k points closest to the origin. Distance should technically be `sqrt(x²+y²)`, but **you don't need the square root for comparisons** — the squared distance `x*x + y*y` preserves the same ordering and is simpler.

k_closest_points — distance

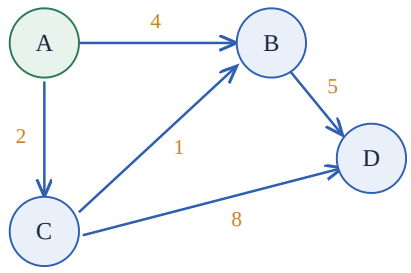
```python
def k_closest_points(points, k):
    if k <= 0 or not points:
        return []
    heap = []
    for point in points:
        x, y = point
        heapq.heappush(heap, (x*x + y*y, point))   # store (squared distance, point)
    result = []
    while len(result) < k and heap:                # two conditions: enough results, OR heap is empty
        distance, point = heapq.heappop(heap)
        result.append(point)
    return result
```

Q Why is the loop condition `while len(result) < k and heap`?

The goal is to collect **k** results, so the primary condition is `len(result) < k`; adding `and heap` prevents crashing on an empty heap when there are fewer than k points total. If you instead wrote something like `while heap: if len(heap)>k: ...`, then once `len(heap) <= k`, the loop body would do nothing while the heap never shrinks — an **infinite loop**.

10 Dijkstra Intro (Weighted Shortest Path)

Book 05's BFS only handles **unweighted** shortest paths (every edge counts as 1 step). When edges have different weights, fewer edges traveled ≠ shorter distance. **Dijkstra** uses a heap to always expand the node with "currently smallest known distance," solving shortest path for non-negative-weight graphs.



A weighted directed graph starting from A

Shortest distances (from A):

A→A = 0

A→C = 2

A→B = 3 (via A→C→B = 2+1, shorter than the direct edge of 4!)

A→D = 8 (A→C→B→D = 2+1+5)

dijkstra — heap picks smallest distance + lazy deletion `dijkstra`

```dijkstra
def dijkstra(graph, start):
    distances = {start: 0}              # known shortest-distance table; start to itself is 0
    heap = [(0, start)]                 # heap stores (distance, node)
    while heap:
        current_distance, current_node = heapq.heappop(heap)  # take the currently-closest one
        if current_distance > distances[current_node]:
            continue                    # a stale, outdated route — skip it (lazy deletion)
        for neighbor, weight in graph.get(current_node, []):  # .get() guards against KeyError
            new_distance = current_distance + weight
            if neighbor not in distances or new_distance < distances[neighbor]:
                distances[neighbor] = new_distance   # found something shorter → update
                heapq.heappush(heap, (new_distance, neighbor))
    return distances

def shortest_distance(graph, start, target):
    return dijkstra(graph, start).get(target)   # unreachable returns None, not a KeyError
```

Q Why does Dijkstra need a heap? Can't BFS's plain queue do the job?

A BFS queue processes nodes in "arrival order," implicitly assuming every edge has the same length. In a weighted graph, you need to always process the node with the **currently smallest distance** next — a plain queue can't do that, but a heap can fetch the minimum in O(log n). Swap BFS's queue for a "heap sorted by distance," and it upgrades into Dijkstra — this is exactly the weighted version of Book 05's `shortest_path`.

Q What's `if current_distance > distances[current_node]: continue` doing?

This is called **lazy deletion**. The same node can get pushed onto the heap multiple times at different distances — for instance the heap might simultaneously contain both `(4,"B")` and `(3,"B")`. Python's heap doesn't offer a convenient way to delete a stale entry from the middle, so they're all left in place; when a **stale** entry eventually gets popped (its distance is larger than the already-known shortest), you just `continue` and skip it.

Three crash-guarding details: ① iterate neighbors with `graph.get(node, [])` so a node with no outgoing edges doesn't raise KeyError; ② check `new_distance < distances[neighbor]` before updating, keeping only the shorter route; ③ fetch results with `distances.get(target)`, which returns `None` instead of KeyError when unreachable. **Limitation:** Dijkstra only works for **non-negative** weights — negative-weight edges need a different algorithm (Bellman-Ford).

11 Bug + FAQ Table

WRONG (OR A POSSIBLE CRASH)	WHY IT'S WRONG	CORRECT
`return heap[0]` (heap could be empty)	Empty heap indexing raises IndexError	First `if not heap: return None`
size-k pop written outside the loop	Pops only once, heap stays bigger than k	Indent the pop **inside** the for loop
`if k < 0`	Misses k==0 (the "0th largest" doesn't exist)	`if k <= 0`
Max heap pop without re-negating	Returns a negative number like -9	`return -heapq.heappop(heap)`
`(task_name, priority)`	Sorts by name, not priority	`(priority, task_name)`
`if heap is None`	An empty heap is [], not None	`if not heap`
`counts.get(n,0)+1` alone on its own line	Computed but never stored back into the dict	`counts[n]=counts.get(n,0)+1`
`heap = numbers` then heapify	Alias — corrupts the original data	`heap = numbers.copy()`
merge doesn't skip empty sub-arrays	`array[0]` crashes on an empty array	`if array:` before taking the front
dijkstra `return distances[target]`	KeyError if unreachable	`distances.get(target)`

> **Closing with the three universal questions:** after writing any heap problem, self-check — ① what exactly am I storing in the heap? ② what does `heap[0]` represent at this point? ③ is what I'm returning the actual thing the problem wants, or an un-unpacked tuple? Answer these three clearly, and this chapter is solid.

12 Exercises + Checklist

Recommended order below — self-check each file against the "three universal questions" as you go.

◆ GROUP 1 · BASICS (EASY)

1. Easy build_min_heap: push `[5,2,8,1,9,3]`, print heap[0] (should be 1).
2. Easy pop_all_sorted / pop_all_descending: get ascending and descending order respectively.
3. Easy Use heapify to find the minimum, then heapify to sort.

◆ GROUP 2 · TOP K & PRIORITY (MEDIUM)

1. Medium find_k_largest / kth_largest: verify against `[5,1,9,3,7,2,8]`.
2. Medium top_k_frequent: `[1,1,1,2,2,3]` top 2 → `[1,2]`.
3. Medium PriorityTaskQueue: add three tasks, verify they pop out in priority order.

◆ GROUP 3 · MULTI-WAY, DISTANCE & SHORTEST PATH (MEDIUM)

1. Medium merge_k_sorted_arrays: including an empty array, `[[],[1,3],[],[2,4]]`.
2. Medium k_closest_points: `[(1,2),(3,4),(0,1),(2,2)]`, find the closest 2.
3. Medium dijkstra: find the distance from A to every node; A→D=8; A→unreachable E = None.

Chapter Checklist

- [ ] heapq defaults to a min heap
- [ ] heap[0] is the current minimum
- [ ] Can't pop an empty heap
- [ ] Negation simulates a max heap
- [ ] Maintain a size-k heap
- [ ] Tuples compare by element 0 first
- [ ] Extract the real answer out of a tuple
- [ ] Handle k<=0 / empty list
- [ ] Dijkstra uses a heap to pick the minimum
- [ ] Lazy deletion skips stale routes
- [ ] Dijkstra only for non-negative weights

Next up → Complexity Analysis (Big-O) • Dynamic Programming • Discrete Math / Induction